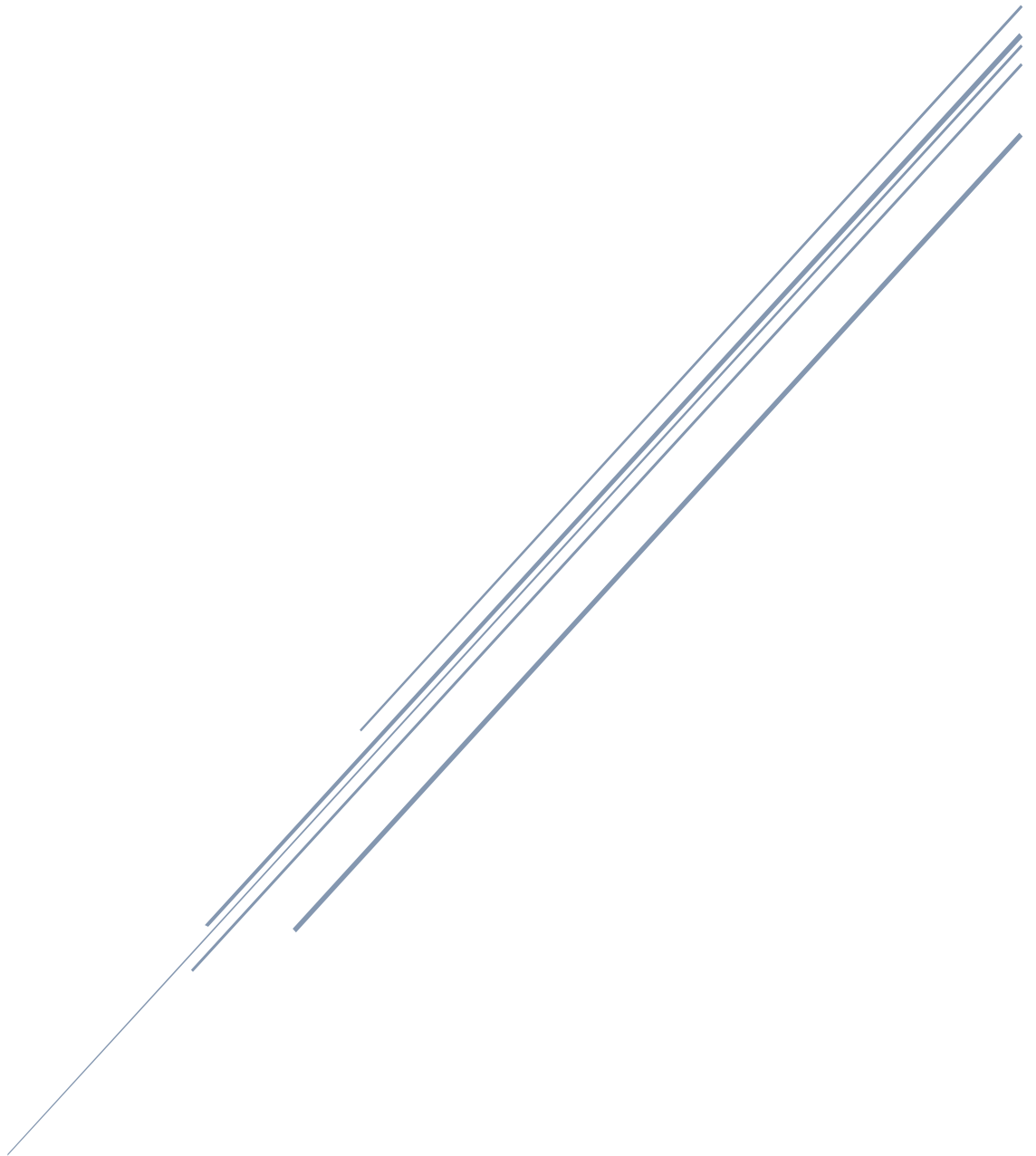


CHAPTER 6

Neural networks



6.1 Shallow neural networks

Shallow neural networks are functions $y = f[x, \phi]$ with parameters ϕ that map multivariate inputs x to multivariate outputs y . An example network $f[x, \phi]$ that maps a scalar input x to a scalar output y and has ten parameters $\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]. \end{aligned}$$

We can break down this calculation into three parts: first, we compute three linear functions of the input data ($\theta_{10} + \theta_{11}x$, $\theta_{20} + \theta_{21}x$, and $\theta_{30} + \theta_{31}x$). Second, we pass the three results through an activation function $a[\cdot]$. Finally, we weight the three resulting activations with ϕ_1 , ϕ_2 , and ϕ_3 , sum them, and add an offset ϕ_0 .

To complete the description, we must define the activation function $a[\cdot]$. There are many possibilities, but the most common choice is the rectified linear unit or ReLU:

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}.$$

This returns the input when it is positive and zero otherwise (figure 6.1).

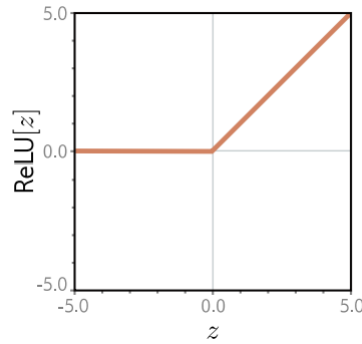


Figure 6.1 Rectified linear unit (ReLU). This activation function returns zero if the input is less than zero and returns the input unchanged otherwise. In other words, it clips negative values to zero.

6.1.1 Neural network intuition

In fact, the previous equation represents a family of continuous piecewise linear functions (Figure 6.2) with up to four linear regions.

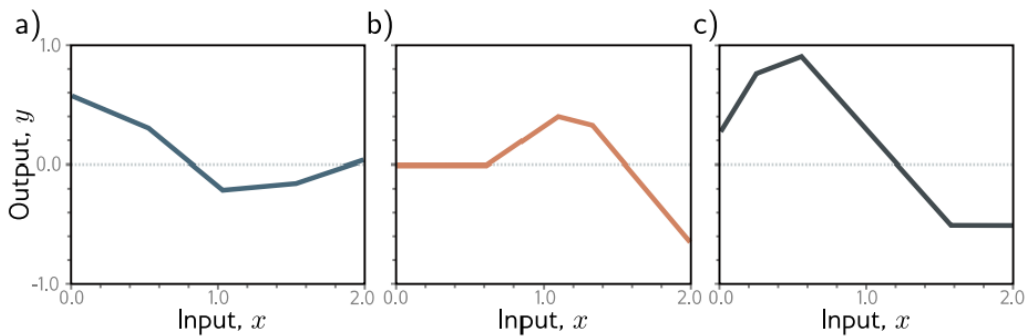


Figure 6.2 Family of piecewise linear functions. a–c) Functions for three different choices of the ten parameters ϕ . In each case, the input/output relation is piecewise linear.

We now break down the previous equation and show why it describes this family. To make this easier to understand, we split the function into two parts. First, we introduce the intermediate quantities:

$$\begin{aligned} h_1 &= a[\theta_{10} + \theta_{11}x] \\ h_2 &= a[\theta_{20} + \theta_{21}x] \\ h_3 &= a[\theta_{30} + \theta_{31}x], \end{aligned}$$

where we refer to h_1 , h_2 , and h_3 as hidden units. Second, we compute the output by combining these hidden units with a linear function:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3.$$

Figure 6.3 shows the flow of computation that creates the function in figure 6.2a. Each hidden unit contains a linear function $\theta_{\cdot 0} + \theta_{\cdot 1}x$ of the input, and that line is clipped by the ReLU function $a[\cdot]$ below zero. The positions where the three lines cross zero become the three “joints” in the final output. The three clipped lines are then weighted by ϕ_1 , ϕ_2 , and ϕ_3 , respectively. Finally, the offset ϕ_0 is added, which controls the overall height of the final function.

Each linear region in figure 3.3j corresponds to a different activation pattern in the hidden units. When a unit is clipped, we refer to it as inactive, and when it is not clipped, we refer to it as active. For example, the shaded region receives contributions from h_1 and h_3 (which are active) but not from h_2 (which is inactive). The slope of each linear region is determined by the original slopes $\theta_{\cdot 1}$ of the active inputs for this region and the weights $\phi_{\cdot}$ that were subsequently applied. For example, the slope in the shaded region is $\theta_{11}\phi_1 + \theta_{31}\phi_3$, where the first term is the slope in panel (g) and the second term is the slope in panel (i).

Each hidden unit contributes one “joint” to the function, so with three hidden units, there can be four linear regions. However, only three of the slopes of these regions are independent; the fourth is either zero (if all the hidden units are inactive in this region) or is a sum of slopes from the other regions.

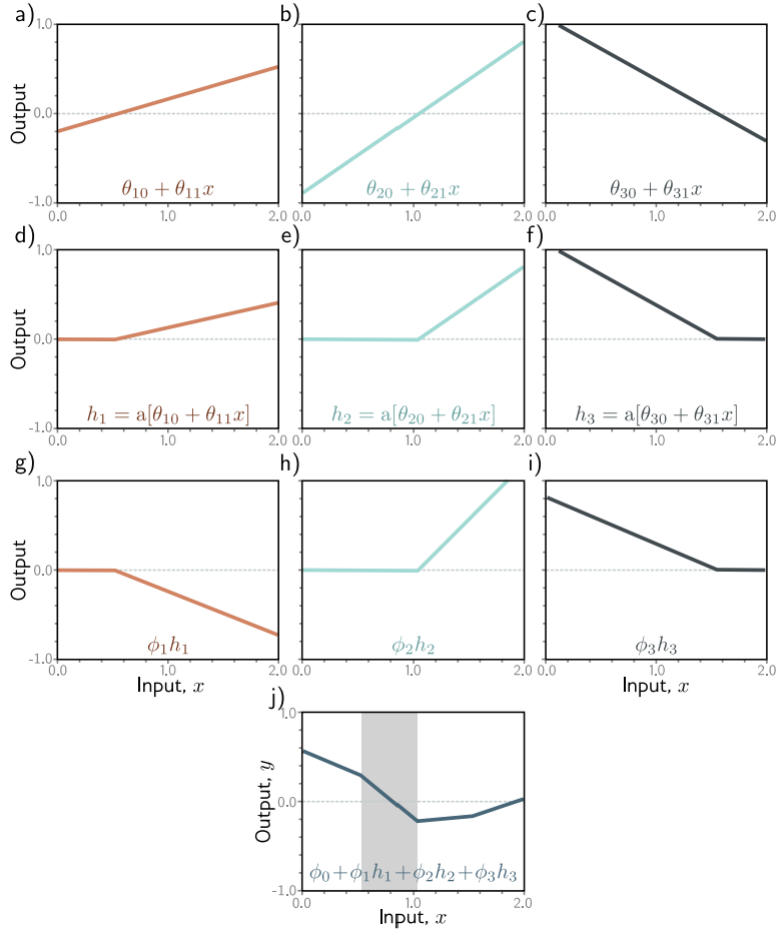


Figure 6.3 Computation for function in figure 6.2a

We have been discussing a neural network with one input, one output, and three hidden units. We visualize this network in figure 6.4a. The input is on the left, the hidden units are in the middle, and the output is on the right. Each connection represents one of the ten parameters. To simplify this representation, we do not typically draw the intercept parameters, so this network is usually depicted as in figure 6.4b.

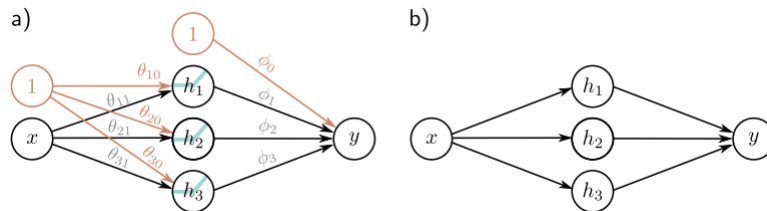


Figure 6.4 Depicting neural networks

6.1.2 Multivariate inputs and outputs

In the above example, the network has a single scalar input x and a single scalar output y . However, this also holds for the more general case where the network maps multivariate inputs $\mathbf{x} = [x_1, x_2, \dots, x_{D_i}]^T$ to multivariate output predictions $\mathbf{y} = [y_1, y_2, \dots, y_{D_o}]^T$. We first explore how to extend the model to predict multivariate outputs. Then we consider multivariate inputs. Finally, we present a general definition of a shallow neural network.

6.1.2.1 Visualizing multivariate outputs

To extend the network to multivariate outputs y , we simply use a different linear function of the hidden units for each output. So, a network with a scalar input x , four hidden units h_1, h_2, h_3 , and h_4 , and a 2D multivariate output $y = [y_1, y_2]^T$ would be defined as:

$$\begin{aligned} h_1 &= a[\theta_{10} + \theta_{11}x] \\ h_2 &= a[\theta_{20} + \theta_{21}x] \\ h_3 &= a[\theta_{30} + \theta_{31}x] \\ h_4 &= a[\theta_{40} + \theta_{41}x], \end{aligned}$$

And

$$\begin{aligned} y_1 &= \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4 \\ y_2 &= \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4. \end{aligned}$$

As we saw in figure 6.3, the “joints” in the piecewise functions depend on where the initial linear functions $\theta_{\cdot 0} + \theta_{\cdot 1}x$ are clipped by the ReLU functions $a[\cdot]$ at the hidden units. Since both outputs y_1 and y_2 are different linear functions of the same four hidden units, the four “joints” in each must be in the same places. However, the slopes of the linear regions and the overall vertical offset can differ (figure 6.5).

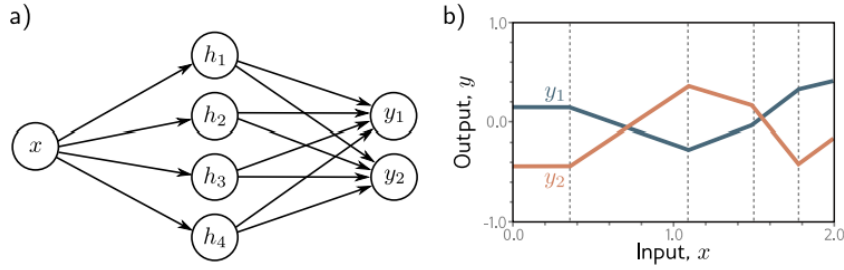


Figure 6.5 Network with one input, four hidden units, and two outputs. a) Visualization of network structure. b) This network produces two piecewise linear functions, $y_1[x]$ and $y_2[x]$. The four “joints” of these functions (at vertical dotted lines) are constrained to be in the same places since they share the same hidden units, but the slopes and overall height may differ.

6.1.2.2 Visualizing multivariate inputs

To cope with multivariate inputs x , we extend the linear relations between the input and the hidden units. So a network with two inputs $x = [x_1, x_2]^T$ and a scalar output y (figure 6.6) might have three hidden units defined by:

$$\begin{aligned} h_1 &= a[\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2] \\ h_2 &= a[\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2] \\ h_3 &= a[\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2], \end{aligned}$$

where there is now one slope parameter for each input. The hidden units are combined to form the output in the usual way:

$$y = \phi_0 + \phi_1h_1 + \phi_2h_2 + \phi_3h_3.$$

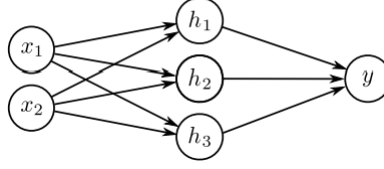


Figure 6.6 Visualization of neural network with 2D multivariate input $x = [x_1, x_2]^T$ and scalar output y .

Figure 6.7 illustrates the processing of this network. Each hidden unit receives a linear combination of the two inputs, which forms an oriented plane in the 3D input/output space. The activation function clips the negative values of these planes to zero. The clipped planes are then recombined in a second linear function to create a continuous piecewise linear surface consisting of convex polygonal regions (figure 3.8j). Each region corresponds to a different activation pattern. For example, in the central triangular region, the first and third hidden units are active, and the second is inactive.

When there are more than two inputs to the model, it becomes difficult to visualize. However, the interpretation is similar. The output will be a continuous piecewise linear function of the input, where the linear regions are now convex polytopes in the multi-dimensional input space.

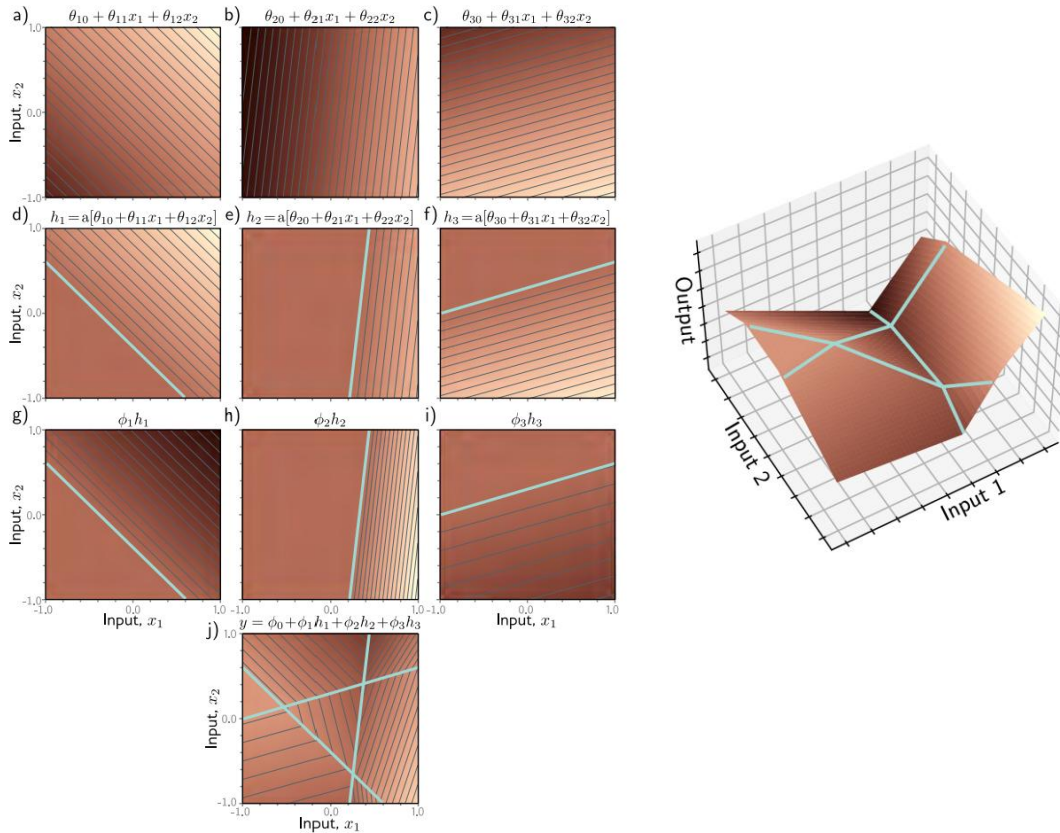


Figure 6.7 Processing in network with two inputs $x = [x_1, x_2]^T$, three hidden units h_1, h_2, h_3 , and one output y . a–c) The input to each hidden unit is a linear function of the two inputs, which corresponds to an oriented plane. Brightness indicates function output. For example, in panel (a), the brightness represents $\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2$. Thin lines are contours. d–f) Each plane is clipped by the ReLU activation function (cyan lines are equivalent to "joints" in figures 3.3d–f). g–i) The clipped planes are then weighted, and j) summed together with an offset that determines the overall height of the surface. The result is a continuous surface made up of convex piecewise linear polygonal regions.

6.1.3 Shallow neural networks: general case

We have described several example shallow networks to help develop intuition about how they work. We now define a general equation for a shallow neural network $y = f[x, \phi]$ that maps a multi-dimensional input $x \in \mathbb{R}^{D_i}$ to a multi-dimensional output $y \in \mathbb{R}^{D_o}$ using $h \in \mathbb{R}^D$ hidden units. Each hidden unit is computed as:

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right],$$

and these are combined linearly to create the output:

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d,$$

where $a[\bullet]$ is a nonlinear activation function. The model has parameters $\phi = \{\theta_{\bullet}, \phi_{\bullet}\}$. Figure 6.8 shows an example with three inputs, three hidden units, and two outputs.

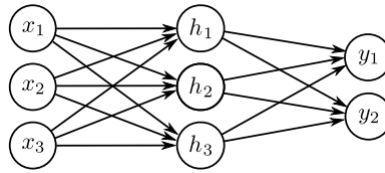


Figure 6.8 Visualization of neural network with three inputs and two outputs

The activation function permits the model to describe nonlinear relations between input and the output, and as such, it must be nonlinear itself; with no activation function, or a linear activation function, the overall mapping from input to output would be restricted to be linear. Many different activation functions have been tried (see figure 6.9), but the most common choice is the ReLU, which has the merit of being easily interpretable. With ReLU activations, the network divides the input space into convex polytopes defined by the intersections of hyperplanes computed by the “joints” in the ReLU functions. Each convex polytope contains a different linear function. The polytopes are the same for each output, but the linear functions they contain can differ.

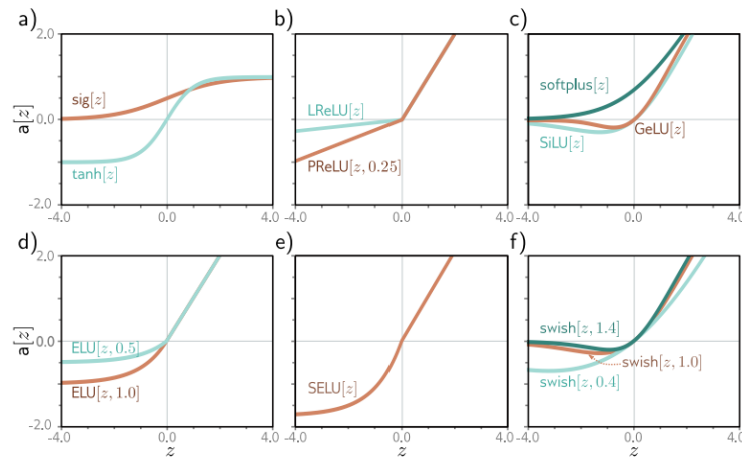


Figure 6.9 Activation functions. a) Logistic sigmoid and tanh functions. b) Leaky ReLU and parametric ReLU with parameter 0.25. c) SoftPlus, Gaussian error linear unit, and sigmoid linear unit. d) Exponential linear unit with parameters 0.5 and 1.0, e) Scaled exponential linear unit. f) Swish with parameters 0.4, 1.0, and 1.4.

6.2 Deep neural networks

6.2.1 Composing neural networks

To gain insight into the behavior of deep neural networks, we first consider composing two shallow networks so the output of the first becomes the input of the second. Consider two shallow networks with three hidden units each (figure 4.1a). The first network takes an input x and returns output y and is defined by:

$$\begin{aligned} h_1 &= a[\theta_{10} + \theta_{11}x] \\ h_2 &= a[\theta_{20} + \theta_{21}x] \\ h_3 &= a[\theta_{30} + \theta_{31}x], \end{aligned}$$

And

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3.$$

The second network takes y as input and returns y' and is defined by:

$$\begin{aligned} h'_1 &= a[\theta'_{10} + \theta'_{11}y] \\ h'_2 &= a[\theta'_{20} + \theta'_{21}y] \\ h'_3 &= a[\theta'_{30} + \theta'_{31}y], \end{aligned}$$

And

$$y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3.$$

With ReLU activations, this model also describes a family of piecewise linear functions. However, the number of linear regions is potentially greater than for a shallow network with six hidden units. To see this, consider choosing the first network to produce three alternating regions of positive and negative slope (figure 6.10b). This means that three different ranges of x are mapped to the same output range $y \in [-1, 1]$, and the subsequent mapping from this range of y to y' is applied three times. The overall effect is that the function defined by the second network is duplicated three times to create nine linear regions. The same principle applies in higher dimensions (figure 6.11).

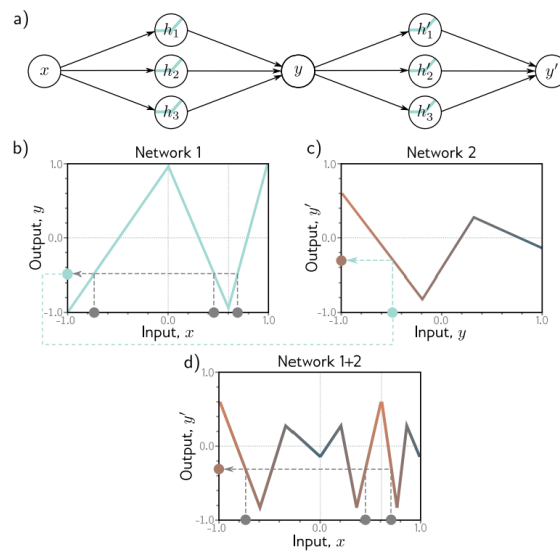


Figure 6.10 Composing two single-layer networks with three hidden units each.

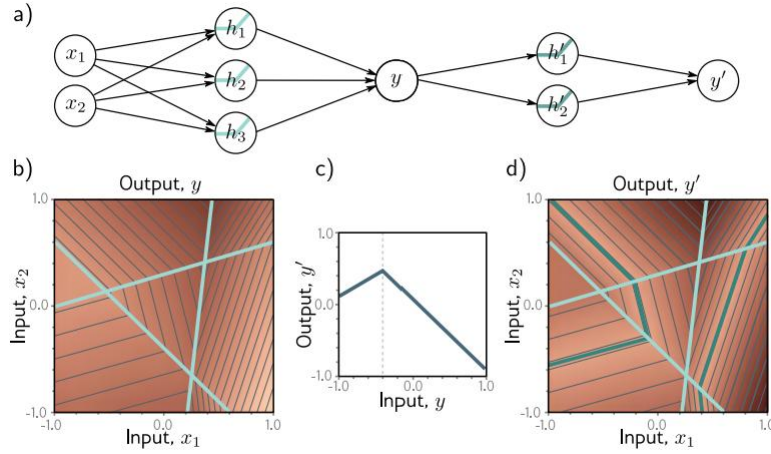


Figure 6.11 Composing neural networks with a 2D input. a) The first network (from figure 6.7) has three hidden units and takes two inputs x_1 and x_2 and returns a scalar output y . This is passed into a second network with two hidden units to produce y' . b) The first network produces a function consisting of seven linear regions, one of which is flat. c) The second network defines a function comprising two linear regions in $y \in [-1, 1]$. d) When these networks are composed, each of the six non-flat regions from the first network is divided into two new regions by the second network to create a total of 13 linear regions.

6.2.2 From composing networks to deep networks

The previous section showed that we could create complex functions by passing the output of one shallow neural network into a second network. We now show that this is a special case of a deep network with two hidden layers. The output of the first network ($y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$) is a linear combination of the activations at the hidden units. The first operations of the second network in which we calculate $\theta'_{10} + \theta'_{11}y$, $\theta'_{20} + \theta'_{21}y$, and $\theta'_{30} + \theta'_{31}y$ are linear in the output of the first network. Applying one linear function to another yields another linear function.

$$\begin{aligned} h'_1 &= a[\theta'_{10} + \theta'_{11}y] = a[\theta'_{10} + \theta'_{11}\phi_0 + \theta'_{11}\phi_1 h_1 + \theta'_{11}\phi_2 h_2 + \theta'_{11}\phi_3 h_3] \\ h'_2 &= a[\theta'_{20} + \theta'_{21}y] = a[\theta'_{20} + \theta'_{21}\phi_0 + \theta'_{21}\phi_1 h_1 + \theta'_{21}\phi_2 h_2 + \theta'_{21}\phi_3 h_3] \\ h'_3 &= a[\theta'_{30} + \theta'_{31}y] = a[\theta'_{30} + \theta'_{31}\phi_0 + \theta'_{31}\phi_1 h_1 + \theta'_{31}\phi_2 h_2 + \theta'_{31}\phi_3 h_3], \end{aligned}$$

which we can rewrite as:

$$\begin{aligned} h'_1 &= a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\ h'_2 &= a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\ h'_3 &= a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3], \end{aligned}$$

where $\psi_{10} = \theta'_{10} + \theta'_{11}\phi_0$, $\psi_{11} = \theta'_{11}\phi_1$, $\psi_{12} = \theta'_{11}\phi_2$ and so on. The result is a network with two hidden layers (figure 6.12).

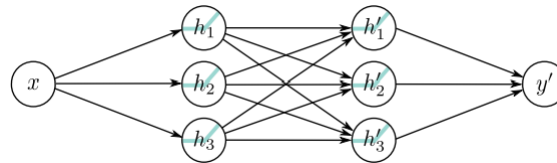


Figure 6.12 Neural network with one input, one output, and two hidden layers, each containing three hidden units.

6.3 Deep neural networks

In the previous section, we showed that composing two shallow networks yields a special case of a deep network with two hidden layers. Now we consider the general case of a deep network with two hidden layers, each containing three hidden units (figure 6.12). The first layer is defined by:

$$\begin{aligned} h_1 &= a[\theta_{10} + \theta_{11}x] \\ h_2 &= a[\theta_{20} + \theta_{21}x] \\ h_3 &= a[\theta_{30} + \theta_{31}x], \end{aligned}$$

the second layer by:

$$\begin{aligned} h'_1 &= a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\ h'_2 &= a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\ h'_3 &= a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3], \end{aligned}$$

and the output by:

$$y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3.$$

Considering these equations leads to another way to think about how the network constructs an increasingly complicated function (figure 6.13):

1. The three hidden units h_1 , h_2 , and h_3 in the first layer are computed as usual by forming linear functions of the input and passing these through ReLU activation functions.
2. The pre-activations at the second layer are computed by taking three new linear functions of these hidden units. At this point, we effectively have a shallow network with three outputs; we have computed three piecewise linear functions with the “joints” between linear regions in the same places.
3. At the second hidden layer, another ReLU function $a[\bullet]$ is applied to each function, which clips them and adds new “joints” to each.
4. The final output is a linear combination of these hidden units.

$$\begin{aligned} y' &= \phi'_0 + \phi'_1 a[\psi_{10} + \psi_{11}a[\theta_{10} + \theta_{11}x] + \psi_{12}a[\theta_{20} + \theta_{21}x] + \psi_{13}a[\theta_{30} + \theta_{31}x]] \\ &\quad + \phi'_2 a[\psi_{20} + \psi_{21}a[\theta_{10} + \theta_{11}x] + \psi_{22}a[\theta_{20} + \theta_{21}x] + \psi_{23}a[\theta_{30} + \theta_{31}x]] \\ &\quad + \phi'_3 a[\psi_{30} + \psi_{31}a[\theta_{10} + \theta_{11}x] + \psi_{32}a[\theta_{20} + \theta_{21}x] + \psi_{33}a[\theta_{30} + \theta_{31}x]], \end{aligned}$$

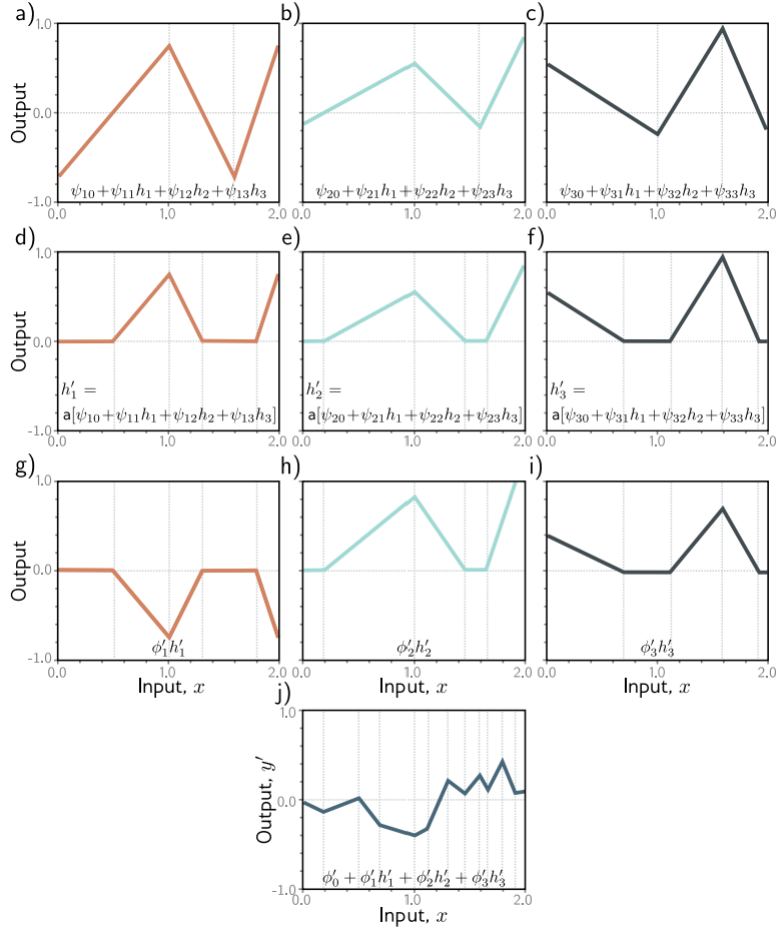


Figure 6.13 Computation for the deep network in figure 6.12.

6.4 Hyperparameters

We can extend the deep network construction to more than two hidden layers; modern networks might have more than a hundred layers with thousands of hidden units at each layer. The number of hidden units in each layer is referred to as the width of the network, and the number of hidden layers as the depth. The total number of hidden units is a measure of the network's capacity.

We denote the number of layers as K and the number of hidden units in each layer as $D_1, D_2, \dots, D_K$. These are examples of hyperparameters. They are quantities chosen before we learn the model parameters (i.e., the slope and intercept terms). For fixed hyperparameters (e.g., $K = 2$ layers with $D_k = 3$ hidden units in each), the model describes a family of functions, and the parameters determine the particular function. Hence, when we also consider the hyperparameters, we can think of neural networks as representing a family of families of functions relating input to output.

6.5 Matrix notation

We have seen that a deep neural network consists of linear transformations alternating with activation functions. We could equivalently describe equations above in matrix notation as:

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \mathbf{a} \left[\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right],$$

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = \mathbf{a} \left[\begin{bmatrix} \psi_{10} \\ \psi_{20} \\ \psi_{30} \end{bmatrix} + \begin{bmatrix} \psi_{11} & \psi_{12} & \psi_{13} \\ \psi_{21} & \psi_{22} & \psi_{23} \\ \psi_{31} & \psi_{32} & \psi_{33} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \right],$$

$$y' = \phi'_0 + [\phi'_1 \quad \phi'_2 \quad \phi'_3] \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix},$$

General case

This notation becomes cumbersome for networks with many layers. Hence, from now on, we will describe the vector of hidden units at layer k as $\mathbf{h}_k$, the vector of biases (intercepts) that contribute to hidden layer $k+1$ as β_k , and the weights (slopes) that are applied to the k^{th} layer and contribute to the $(k+1)^{\text{th}}$ layer as Ω_k . A general deep network $y = f[\mathbf{x}, \phi]$ with K layers can now be written as:

$$\begin{aligned} \mathbf{h}_1 &= \mathbf{a}[\beta_0 + \Omega_0 \mathbf{x}] \\ \mathbf{h}_2 &= \mathbf{a}[\beta_1 + \Omega_1 \mathbf{h}_1] \\ \mathbf{h}_3 &= \mathbf{a}[\beta_2 + \Omega_2 \mathbf{h}_2] \\ &\vdots \\ \mathbf{h}_K &= \mathbf{a}[\beta_{K-1} + \Omega_{K-1} \mathbf{h}_{K-1}] \\ \mathbf{y} &= \beta_K + \Omega_K \mathbf{h}_K. \end{aligned}$$

The parameters ϕ of this model comprise all of these weight matrices and bias vectors $\phi = \{\beta_k, \Omega_k\}_{k=0}^K$. If the k^{th} layer has D_k hidden units, then the bias vector β_{k-1} will be of size D_k . The last bias vector β_K has the size D_o of the output. The first weight matrix Ω_0 has size $D_1 \times D_i$ where D_i is the size of the input. The last weight matrix Ω_K is $D_o \times D_K$, and the remaining matrices Ω_k are $D_{k+1} \times D_k$ (figure 6.14). We can equivalently write the network as a single function:

$$\mathbf{y} = \beta_K + \Omega_K \mathbf{a} [\beta_{K-1} + \Omega_{K-1} \mathbf{a} [\dots \beta_2 + \Omega_2 \mathbf{a} [\beta_1 + \Omega_1 \mathbf{a} [\beta_0 + \Omega_0 \mathbf{x}] \dots]]].$$

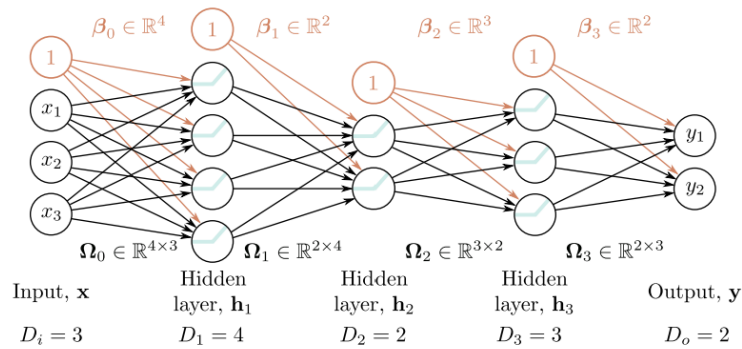


Figure 6.14 Matrix notation for network with $D_i = 3$ -dimensional input $\mathbf{x}$, $D_o = 2$ -dimensional output $\mathbf{y}$, and $K = 3$ hidden layers $\mathbf{h}_1$, $\mathbf{h}_2$, and $\mathbf{h}_3$ of dimensions $D_1 = 4$, $D_2 = 2$, and $D_3 = 3$ respectively. The weights are stored in matrices Ω_k that multiply the activations from the preceding layer to create the pre-activations at the subsequent layer. For example, the weight matrix Ω_1 that computes the pre-activations at $\mathbf{h}_2$ from the activations at $\mathbf{h}_1$ has dimension 2×4 . It is applied to the four hidden units in layer one and creates the inputs to the two hidden units at layer two. The biases are stored in vectors β_k and have the dimension of the layer into which they feed. For example, the bias vector β_2 is length three because layer $\mathbf{h}_3$ contains three hidden units.